

Quiz — 1.1.1 Structure and Function of the Processor

Name: _____ Date: _ **Mark:** ____ / 25

OCR H446 · Component 01 · 1.1.1

Section A — Multiple choice (8 marks)

1. Which register always holds the address of the **next** instruction to be fetched? A. MAR B. MDR C. PC D. CIR Your answer: ☐
2. Which register holds the **data or instruction** that has just been fetched from, or is about to be written to, memory? A. MAR B. MDR C. ACC D. CIR Your answer: ☐
3. The **address bus** is best described as: A. Bidirectional, carrying data and instructions B. Unidirectional, carrying memory addresses from the CPU to memory C. Bidirectional, carrying control signals D. Unidirectional, carrying data from memory to the CPU Your answer: ☐
4. During the **fetch** stage, immediately after the contents of the PC are copied to the MAR, what happens next? A. The instruction is decoded into opcode and operand B. The PC is reset to zero C. A read signal is sent on the control bus and the instruction returns via the data bus to the MDR D. The ALU performs the calculation and stores the result in the ACC Your answer: ☐
5. What is the primary purpose of the **Control Unit (CU)**? A. To perform arithmetic and logical operations B. To store the immediate result of calculations C. To decode instructions and generate control signals that coordinate the CPU D. To provide fast storage of frequently used data Your answer: ☐
6. **Cache memory** improves performance because it is: A. Part of main RAM that has been reserved B. Small, very fast memory on or near the CPU holding frequently/recently used data C. A region of secondary storage used when RAM is full D. A register inside the ALU Your answer: ☐
7. Which statement about adding **more cores** is correct? A. More cores always doubles performance B. More cores only improve performance if the task can be parallelised and the software supports it C. More cores reduce clock speed proportionally D. More cores remove the need for cache Your answer: ☐
8. **Pipelining** increases performance by: A. Increasing the clock speed automatically B. Overlapping the fetch, decode and execute stages of consecutive instructions C. Doubling the width of the data bus D. Storing more instructions in the accumulator Your answer: ☐

Section B — Short answer (12 marks)

9. State the full name of each register and give its role. [3]

- MAR: _____
- MDR: _____
- CIR: _____

10. Describe the role of the **ALU** within the processor. [2]

11. A CPU has a **16-bit address bus**. Calculate the maximum number of memory locations that can be addressed. Show your working. **[2]**

12. Explain how increasing the **clock speed** can improve CPU performance, and state **one** factor that limits how high clock speed can go. **[2]**

13. Explain the difference between the **L1** and **L3** cache in terms of speed and size, and state the order in which the CPU checks them. **[2]**

14. A **jump (branch)** instruction is executed. Explain what happens to the PC and why this can cause a problem for a pipelined processor. **[1]**

Section C — Exam-style question (5 marks)

15. A games developer is comparing two processors. Processor X has a higher clock speed but a single core; Processor Y has a lower clock speed but four cores and a larger cache.

Discuss the factors the developer should consider when deciding which processor will give better performance for their game. **(AO2/AO3) [5]**

Answer key

Section A (8 marks — 1 each)

1. **C** — The Program Counter always holds the address of the next instruction; it is incremented each cycle.
2. **B** — The Memory Data Register holds data/instruction just fetched or about to be written. (Remember: MAR = address, MDR = data.)
3. **B** — The address bus is unidirectional, CPU → memory only.
4. **C** — After PC → MAR, a read signal is placed on the control bus and the instruction returns via the data bus into the MDR.
5. **C** — The CU decodes instructions and generates control signals to synchronise and direct the CPU.
6. **B** — Cache is small, very fast memory on/near the CPU (not part of RAM) holding frequently/recently used data.
7. **B** — More cores help only if the task is parallelisable and software supports it.
8. **B** — Pipelining overlaps fetch/decode/execute of consecutive instructions to raise throughput.

Section B (12 marks)

9. **[3]** — 1 mark each (full name + role): - MAR = **Memory Address Register** — holds the address of the location to be read from / written to. - MDR = **Memory Data Register** — holds the data/instruction just fetched, or about to be written, to memory. - CIR = **Current Instruction Register** — holds the current instruction while it is decoded/executed.
10. **[2]** — 1 mark each: Performs **arithmetic** operations (+ − × ÷); performs **logic** operations / comparisons (AND, OR, NOT, compare) and shifts; (result is placed in the ACC). Any two distinct points.
11. **[2]** — 2^{16} (1 mark for method 2^n) = **65,536 locations** (1 mark for correct answer).
12. **[2]** — 1 mark: higher clock speed = more pulses/cycles per second, so more fetch–decode–execute cycles completed per second / more instructions processed per second. 1 mark: limited by **heat / power consumption** (overheating).
13. **[2]** — 1 mark: L1 is **smaller and faster** (on the core); L3 is **larger but slower** (often shared) — though still faster than RAM. 1 mark: checked in order **L1 → L2 → L3 → RAM**.
14. **[1]** — A **new address is written into the PC** (so execution continues elsewhere); the instructions already prefetched into the pipeline must be **flushed/discarded** (branch hazard), wasting cycles.

Section C (5 marks)

15. **[5]** — Level-based / point marking, award up to 5 for a balanced, applied discussion. Indicative content: - Higher clock speed (X) means each individual core completes more FDE cycles per second — good for the **sequential / single-threaded** parts of a game (e.g. game logic, physics dependencies). - More cores (Y) allow **parallel processing** — multiple tasks (rendering prep, AI, audio, physics) run simultaneously — but **only if the game engine is written to use multiple threads**. - Larger cache (Y) reduces slow RAM fetches (fewer cache misses), improving throughput for frequently accessed data/instructions. - Trade-off: if the game is poorly multi-threaded, X's higher clock speed may win; modern game engines are increasingly parallel, favouring Y. - Justified conclusion linking the choice to the workload (e.g. "Y is likely better for a modern, multi-threaded engine because..."). Max 4 if no justified judgement/conclusion is made.

Total: 8 + 12 + 5 = 25